

```
1 //#####
2 // FILE:   LABstarter_main.c
3 //
4 // TITLE:   Lab Starter
5 //#####
6
7 // Included Files
8 #include <stdio.h>
9 #include <stdlib.h>
10 #include <stdarg.h>
11 #include <string.h>
12 #include <math.h>
13 #include <limits.h>
14 #include "F28x_Project.h"
15 #include "driverlib.h"
16 #include "device.h"
17 #include "F28379dSerial.h"
18 #include "song.h"
19 #include "dsp.h"
20 #include "fpu32/fpu_rfft.h"
21
22 #define PI          3.1415926535897932384626433832795
23 #define TWOPI       6.283185307179586476925286766559
24 #define HALFPI      1.5707963267948966192313216916398
25 // The Launchpad's CPU Frequency set to 200 you should not change this value
26 #define LAUNCHPAD_CPU_FREQUENCY 200
27
28
29 // Interrupt Service Routines predefinition
30 __interrupt void cpu_timer0_isr(void);
31 __interrupt void cpu_timer1_isr(void);
32 __interrupt void cpu_timer2_isr(void);
33 __interrupt void SWI_isr(void);
34
35 // Count variables
36 uint32_t numTimer0calls = 0;
37 uint32_t numSWIcalls = 0;
38 extern uint32_t numRXA;
39 uint16_t UARTPrint = 0;
40
41 // sj our global vars
42 int32_t numTimer2calls = 0;
43 int16_t temp = 0;
44 int32_t countWhenNotPressed = 0;
45
46 // vars for breakpoint exercise
47 float x1= 6.0;
48 float x2= 2.3;
49 float x3= 7.3;
50 float x4= 7.1;
51
52 // sjew defining new funcs
53 void SetLEDsOnOff(int16_t LEDvalue);
54 int16_t ReadSwitches(void);
55
56 void main(void)
57 {
```

```
58 // PLL, WatchDog, enable Peripheral Clocks
59 // This example function is found in the F2837xD_SysCtrl.c file.
60 InitSysCtrl();
61
62 InitGpio();
63
64 // Blue LED on LaunchPad
65 GPIO_SetupPinMux(31, GPIO_MUX_CPU1, 0);
66 GPIO_SetupPinOptions(31, GPIO_OUTPUT, GPIO_PUSHPULL);
67 GpioDataRegs.GPASET.bit.GPIO31 = 1;
68
69 // Red LED on LaunchPad
70 GPIO_SetupPinMux(34, GPIO_MUX_CPU1, 0);
71 GPIO_SetupPinOptions(34, GPIO_OUTPUT, GPIO_PUSHPULL);
72 GpioDataRegs.GPBSET.bit.GPIO34 = 1;
73
74 // LED1 and PWM Pin
75 GPIO_SetupPinMux(22, GPIO_MUX_CPU1, 0);
76 GPIO_SetupPinOptions(22, GPIO_OUTPUT, GPIO_PUSHPULL);
77 GpioDataRegs.GPACLEAR.bit.GPIO22 = 1;
78
79 // LED2
80 GPIO_SetupPinMux(94, GPIO_MUX_CPU1, 0);
81 GPIO_SetupPinOptions(94, GPIO_OUTPUT, GPIO_PUSHPULL);
82 GpioDataRegs.GPCCLEAR.bit.GPIO94 = 1;
83
84 // LED3
85 GPIO_SetupPinMux(95, GPIO_MUX_CPU1, 0);
86 GPIO_SetupPinOptions(95, GPIO_OUTPUT, GPIO_PUSHPULL);
87 GpioDataRegs.GPCCLEAR.bit.GPIO95 = 1;
88
89 // LED4
90 GPIO_SetupPinMux(97, GPIO_MUX_CPU1, 0);
91 GPIO_SetupPinOptions(97, GPIO_OUTPUT, GPIO_PUSHPULL);
92 GpioDataRegs.GPDCLEAR.bit.GPIO97 = 1;
93
94 // LED5
95 GPIO_SetupPinMux(111, GPIO_MUX_CPU1, 0);
96 GPIO_SetupPinOptions(111, GPIO_OUTPUT, GPIO_PUSHPULL);
97 GpioDataRegs.GPDCLEAR.bit.GPIO111 = 1;
98
99 // LS7366#1 CS
100 GPIO_SetupPinMux(130, GPIO_MUX_CPU1, 0);
101 GPIO_SetupPinOptions(130, GPIO_OUTPUT, GPIO_PUSHPULL);
102 GpioDataRegs.GPECLEAR.bit.GPIO130 = 1;
103
104 // LS7366#2 CS
105 GPIO_SetupPinMux(131, GPIO_MUX_CPU1, 0);
106 GPIO_SetupPinOptions(131, GPIO_OUTPUT, GPIO_PUSHPULL);
107 GpioDataRegs.GPECLEAR.bit.GPIO131 = 1;
108
109 // LS7366#3 CS
110 GPIO_SetupPinMux(25, GPIO_MUX_CPU1, 0);
111 GPIO_SetupPinOptions(25, GPIO_OUTPUT, GPIO_PUSHPULL);
112 GpioDataRegs.GPACLEAR.bit.GPIO25 = 1;
113
114 // LS7366#4 CS
```

```
115     GPIO_SetupPinMux(26, GPIO_MUX_CPU1, 0);
116     GPIO_SetupPinOptions(26, GPIO_OUTPUT, GPIO_PUSHPULL);
117     GpioDataRegs.GPACLEAR.bit.GPIO26 = 1;
118
119     // WIZNET RST
120     GPIO_SetupPinMux(27, GPIO_MUX_CPU1, 0);
121     GPIO_SetupPinOptions(27, GPIO_OUTPUT, GPIO_PUSHPULL);
122     GpioDataRegs.GPACLEAR.bit.GPIO27 = 1;
123
124     //PushButton 1
125     GPIO_SetupPinMux(157, GPIO_MUX_CPU1, 0);
126     GPIO_SetupPinOptions(157, GPIO_INPUT, GPIO_PULLUP);
127
128     //PushButton 2
129     GPIO_SetupPinMux(158, GPIO_MUX_CPU1, 0);
130     GPIO_SetupPinOptions(158, GPIO_INPUT, GPIO_PULLUP);
131
132     //PushButton 3
133     GPIO_SetupPinMux(159, GPIO_MUX_CPU1, 0);
134     GPIO_SetupPinOptions(159, GPIO_INPUT, GPIO_PULLUP);
135
136     //PushButton 4
137     GPIO_SetupPinMux(160, GPIO_MUX_CPU1, 0);
138     GPIO_SetupPinOptions(160, GPIO_INPUT, GPIO_PULLUP);
139
140     //SPIRAM CS Chip Select
141     GPIO_SetupPinMux(19, GPIO_MUX_CPU1, 0);
142     GPIO_SetupPinOptions(19, GPIO_OUTPUT, GPIO_PUSHPULL);
143     GpioDataRegs.GPASET.bit.GPIO19 = 1;
144
145     //F28027 CS
146     GPIO_SetupPinMux(29, GPIO_MUX_CPU1, 0);
147     GPIO_SetupPinOptions(29, GPIO_OUTPUT, GPIO_PUSHPULL);
148     GpioDataRegs.GPASET.bit.GPIO29 = 1;
149
150     //MPU9250 CS Chip Select
151     GPIO_SetupPinMux(66, GPIO_MUX_CPU1, 0);
152     GPIO_SetupPinOptions(66, GPIO_OUTPUT, GPIO_PUSHPULL);
153     GpioDataRegs.GPCSET.bit.GPIO66 = 1;
154
155     //WIZNET CS Chip Select
156     GPIO_SetupPinMux(125, GPIO_MUX_CPU1, 0);
157     GPIO_SetupPinOptions(125, GPIO_OUTPUT, GPIO_PUSHPULL);
158     GpioDataRegs.GPDSET.bit.GPIO125 = 1;
159
160     // Clear all interrupts and initialize PIE vector table:
161     // Disable CPU interrupts
162     DINT;
163
164     // Initialize the PIE control registers to their default state.
165     // The default state is all PIE interrupts disabled and flags
166     // are cleared.
167     // This function is found in the F2837xD_PieCtrl.c file.
168     InitPieCtrl();
169
170     // Disable CPU interrupts and clear all CPU interrupt flags:
171     IER = 0x0000;
```

```
172     IFR = 0x0000;
173
174     // Initialize the PIE vector table with pointers to the shell Interrupt
175     // Service Routines (ISR).
176     // This will populate the entire table, even if the interrupt
177     // is not used in this example. This is useful for debug purposes.
178     // The shell ISR routines are found in F2837xD_DefaultIsr.c.
179     // This function is found in F2837xD_PieVect.c.
180     InitPieVectTable();
181
182     // Interrupts that are used in this example are re-mapped to
183     // ISR functions found within this project
184     EALLOW; // This is needed to write to EALLOW protected registers
185     PieVectTable.TIMER0_INT = &cpu_timer0_isr;
186     PieVectTable.TIMER1_INT = &cpu_timer1_isr;
187     PieVectTable.TIMER2_INT = &cpu_timer2_isr;
188     PieVectTable.SCIA_RX_INT = &RXAINT_recv_ready;
189     PieVectTable.SCIB_RX_INT = &RXBINT_recv_ready;
190     PieVectTable.SCIC_RX_INT = &RXCINT_recv_ready;
191     PieVectTable.SCID_RX_INT = &RXDINT_recv_ready;
192     PieVectTable.SCIA_TX_INT = &TXAINT_data_sent;
193     PieVectTable.SCIB_TX_INT = &TXBINT_data_sent;
194     PieVectTable.SCIC_TX_INT = &TXCINT_data_sent;
195     PieVectTable.SCID_TX_INT = &TXDINT_data_sent;
196
197     PieVectTable.EMIF_ERROR_INT = &SWI_isr;
198     EDIS; // This is needed to disable write to EALLOW protected registers
199
200
201     // Initialize the CpuTimers Device Peripheral. This function can be
202     // found in F2837xD_CpuTimers.c
203     InitCpuTimers();
204
205     // Configure CPU-Timer 0, 1, and 2 to interrupt every given period:
206     // 200MHz CPU Freq, Period (in uSeconds)
207     ConfigCpuTimer(&CpuTimer0, LAUNCHPAD_CPU_FREQUENCY, 10000);
208     ConfigCpuTimer(&CpuTimer1, LAUNCHPAD_CPU_FREQUENCY, 20000);
209     ConfigCpuTimer(&CpuTimer2, LAUNCHPAD_CPU_FREQUENCY, 1000);
210
211     // Enable CpuTimer Interrupt bit TIE
212     // sjew disabled other interrupts
213 //     CpuTimer0Regs.TCR.all = 0x4000;
214 //     CpuTimer1Regs.TCR.all = 0x4000;
215     CpuTimer2Regs.TCR.all = 0x4000;
216
217     init_serialSCIA(&SerialA,115200);
218     init_serialSCIB(&SerialB,19200);
219 //     init_serialSCIC(&SerialC,115200);
220 //     init_serialSCID(&SerialD,115200);
221
222     // Enable CPU int1 which is connected to CPU-Timer 0, CPU int13
223     // which is connected to CPU-Timer 1, and CPU int 14, which is connected
224     // to CPU-Timer 2: int 12 is for the SWI.
225     IER |= M_INT1;
226     IER |= M_INT8; // SCIC SCID
227     IER |= M_INT9; // SCIA
228     IER |= M_INT12;
```

```

229     IER |= M_INT13;
230     IER |= M_INT14;
231
232     // Enable TINT0 in the PIE: Group 1 interrupt 7
233     PieCtrlRegs.PIEIER1.bit.INTx7 = 1;
234     // Enable SWI in the PIE: Group 12 interrupt 9
235     PieCtrlRegs.PIEIER12.bit.INTx9 = 1;
236
237     // Enable global Interrupts and higher priority real-time debug events
238     EINT; // Enable Global interrupt INTM
239     ERTM; // Enable Global realtime interrupt DBGEM
240
241
242     // IDLE loop. Just sit and loop forever (optional):
243     while(1)
244     {
245         if (UARTPrint == 1 ) {
246             // Normally on the Robot Car we only use the below UART_printfLine functions to
write to the
247             // on board LCD screen. This below serial_printf is only used in lab 1 to print
to a serial
248             // terminal over a USB cable like you will for your Homeworks.
249             //serial_printf(&SerialA,"Num Timer2:%ld Num SerialRX: %ld\r
\n",CpuTimer2.InterruptCount,numRXA);
250
251             //IMPORTANT!! %ld is for an int32_t. To print an int16_t use %d
252             UART_printfLine(1,"T2 %ld|3x : %ld", numTimer2calls, numTimer2calls*3);
253             UART_printfLine(2,"switch val %d, %x", temp, temp );
254             UARTPrint = 0;
255         }
256     }
257 }
258
259
260 // SWI_isr, Using this interrupt as a Software started interrupt
261 __interrupt void SWI_isr(void) {
262
263     // These three lines of code allow SWI_isr, to be interrupted by other interrupt functions
264     // making it lower priority than all other Hardware interrupts.
265     PieCtrlRegs.PIEACK.all = PIEACK_GROUP12;
266     asm("    NOP"); // Wait one cycle
267     EINT; // Clear INTM to enable interrupts
268
269
270
271     // Insert SWI ISR Code here.....
272
273
274     numSWIcalls++;
275
276     DINT;
277 }
278 }
279
280 // cpu_timer0_isr - CPU Timer0 ISR
281 __interrupt void cpu_timer0_isr(void)
282 {

```

```
283     CpuTimer0.InterruptCount++;
284
285     numTimer0calls++;
286
287 //     if ((numTimer0calls%50) == 0) {
288 //         PieCtrlRegs.PIEIFR12.bit.INTx9 = 1; // Manually cause the interrupt for the SWI
289 //     }
290
291     if ((numTimer0calls%5) == 0) {
292         // Blink LaunchPad Red LED
293         GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1;
294     }
295
296
297     // Acknowledge this interrupt to receive more interrupts from group 1
298     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
299 }
300
301 // cpu_timer1_isr - CPU Timer1 ISR
302 __interrupt void cpu_timer1_isr(void)
303 {
304
305     CpuTimer1.InterruptCount++;
306 }
307
308 // cpu_timer2_isr CPU Timer2 ISR
309 __interrupt void cpu_timer2_isr(void)
310 {
311     // Blink LaunchPad Blue LED
312     GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1;
313
314     CpuTimer2.InterruptCount++;
315
316     if ((CpuTimer2.InterruptCount % 10) == 0) {
317         UARTPrint = 1;
318     }
319
320     // sjew increments count
321     numTimer2calls++;
322
323
324     // sjew update temp with current switch values and write to leds (commented out)
325     temp = ReadSwitches();
326
327     //SetLEDsOnOff(temp);
328
329     // testing endianness
330     //uint32_t temp1 = 0xFF000001;
331     //SetLEDsOnOff(temp1);
332
333     // sjew every 100 calls with 1ms period
334     if (numTimer2calls % 100 == 0){
335         // sjew Leds count in binary when button 2 and 3 are not pressed
336         SetLEDsOnOff(countWhenNotPressed);
337
338         // checking if button 2 and 3 are not both pressed
339         if (!((temp & 0x6) == 0x6)){
```

```
340         countWhenNotPressed++;
341     }
342     UARTPrint = 1;
343 }
344
345 // sjew testing breakpoints
346 x4 = x3 + 2.0;
347 x3 = x4 + 1.3;
348 x1 = 9*x2;
349 x2 = 34*x3;
350
351
352 }
353
354
355 // sjew sets or clears onboard leds based of lowest 5 bits of the LEDvalue
356 void SetLEDsOnOff(int16_t LEDvalue){
357
358     // sj led 1
359     if ((LEDvalue & 0x1) == 0x1) {
360         GpioDataRegs.GPASET.bit.GPIO22 = 1;
361     }
362     else{
363         GpioDataRegs.GPACLEAR.bit.GPIO22 = 1;
364     }
365
366     // sj led 2
367     if ((LEDvalue & 0x2) == 0x2) {
368         GpioDataRegs.GPCSET.bit.GPIO94 = 1;
369     }
370     else{
371         GpioDataRegs.GPCCLEAR.bit.GPIO94 = 1;
372     }
373
374     // sj led 3
375     if ((LEDvalue & 0x4) == 0x4) {
376         GpioDataRegs.GPCSET.bit.GPIO95 = 1;
377     }
378     else{
379         GpioDataRegs.GPCCLEAR.bit.GPIO95 = 1;
380     }
381
382     // sj led 4
383     if ((LEDvalue & 0x8) == 0x8) {
384         GpioDataRegs.GPDSET.bit.GPIO97 = 1;
385     }
386     else{
387         GpioDataRegs.GPDCLEAR.bit.GPIO97 = 1;
388     }
389
390     // sj led 5
391     if ((LEDvalue & 0x10) == 0x10) {
392         GpioDataRegs.GPDSET.bit.GPIO111 = 1;
393     }
394     else{
395         GpioDataRegs.GPDCLEAR.bit.GPIO111 = 1;
396     }
```

```
397
398 }
399
400 // sjew returns bit mask of buttons inputs
401 int16_t ReadSwitches(void){
402     uint16_t sw = 0;
403
404     // button 1
405     if ( GpioDataRegs.GPEDAT.bit.GPIO157 == 0) {
406         sw |= 0x1;
407     }
408
409     // button 2
410     if ( GpioDataRegs.GPEDAT.bit.GPIO158 == 0) {
411         sw |= 0x2;
412     }
413
414     // button 3
415     if ( GpioDataRegs.GPEDAT.bit.GPIO159 == 0) {
416         sw |= 0x4;
417     }
418
419     // button 4
420     if ( GpioDataRegs.GPFDAT.bit.GPIO160 == 0) {
421         sw |= 0x8;
422     }
423
424     return sw;
425 }
426
```